



FACULTY OF ENGINEERING & TECHNOLOGY
ELECTRICAL AND COMPUTER ENGINEERING
DEPARTMENT
ADVANCED DIGITAL DESIGN ENCS3310
COURSE PROJECT

student name: Rawand Radi

Student Id : 1221054

Instructor name : Elias Khalil

Section: 1

Date : 23th - Dec - 2024.

Abstract

The aim of this project is to build a structural comparator that compare between two 6-bit numbers , the comparator will determine whether the two inputs (A and B) are equal, greater, or smaller. A selection input (S) is included to switch between signed and unsigned comparisons. The comparator built using a library of basic gates,with fixed delays.

To achieve synchronous operation, registers will be added to the inputs and outputs,also will calculate the maximum latency of the circuit and determine the maximum clock frequency it can support,This design ensures the circuit works correctly and is fast, while helping to understand how circuits are built and tested.

Table of Content

Abstratct	I
Table of Content	II
Table of Figures	III
Theory	1
Unsigned Comparator	1
Signed Comparator	1
Synchronous Circuit	1
Design Philosophy	2
Unsigned Comparator	2
Signed comparison	3
Multiplexer	4
Making the Circuit Synchronous Using Registers	5
Comparator full design	5
Simulation Results	6
Conclusion and Future works	8

Table of Figures

Figure 1 :Unsigned Comparator	3
Figure 2 :Signed Comparator	4
Figure 3 :Full adder	4
Figure 4 :2x1 multiplexer	5
Figure 5 : registers	5
Figure 6 : comparator block diagram	6
Figure 7 :Waveform Result	7
Figure 8 :Design error dedected	7

Theory

A comparator is a digital circuit used to compare two binary numbers and determine their relationship—whether they are equal, greater, or smaller. There are two parts of the comparator: one handles signed numbers, and the other handles unsigned numbers. The selection input is used to switch between the parts.

Unsigned Comparator

Magnitude comparator is a type of combinational circuit. It basically compares two binary numbers and determines their relative magnitude. It gives output whether one number is greater than the other, or less than or equal. These comparators are used in digital systems, such as for sorting networks, and decision-making circuits to handle numerical comparisons perfectly without any error.

Signed Comparator

A digital circuit that compares two signed numbers by performing signed subtraction between the two inputs. Based on the result of the subtraction, the comparator determines whether the first number (A) is equal to, greater than, or less than the second number (B).

Synchronous Circuit

A synchronous circuit operates based on a global clock signal, which ensures that all changes in the circuit state occur in a coordinated manner. For a synchronous comparator, this means adding registers at the inputs and outputs to synchronize data flow and operations with the clock.

Design Philosophy

The design of the comparator is based on a structural approach, where all combinational logic circuits are constructed using basic gates. The main design includes two types of comparators to handle signed and unsigned numbers, a multiplexer (Mux) to switch between them, and registers to synchronize the circuit. Additionally, a delay was assigned to each gate to account for timing considerations in the design.

Unsigned Comparator

The logic for the comparison (Equal, Greater, and Smaller) was built using a combination of AND, OR, XOR, and NOT gates. Only one of the outputs will be 1 at a time, while the others will be 0. The 'Greater than' output for each bit (GT) is obtained using the following logic expression $GT_i = A \text{ AND } \sim B$.

The **Equal** output for one bit is computed using $EQ_i = A \text{ AND } B + \sim A \text{ AND } \sim B$ which checks if the two bits are equal. The **Less than** output for one bit is determined by $LT_i = \sim A \text{ AND } B$ which checks if A is less than B.

These individual bitwise comparisons are then extended to handle the full 6-bit inputs, with the outputs being propagated through each bit to produce the final comparison results.

$$GT = GT_n + (EQ_n \text{ AND } GT_{n-1}) + (EQ_n \text{ AND } EQ_{n-1} \text{ AND } GT_{n-2}) + \dots + (\prod_{i=1}^n EQ_i \text{ AND } GT_0).$$

$$LT = LT_n + (EQ_n \text{ AND } LT_{n-1}) + (EQ_n \text{ AND } EQ_{n-1} \text{ AND } LT_{n-2}) + \dots + (\prod_{i=1}^n EQ_i \text{ AND } LT_0).$$

$$EQ = \prod_{i=1}^n EQ_i.$$

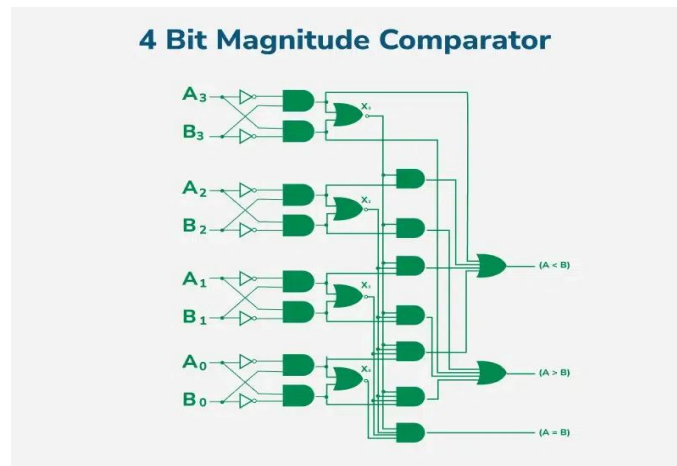


Figure 1: Unsigned Comparator

Signed comparison

Another method used to handle signed numbers is through signed subtraction. In this approach, full adders are used with the initial carry-in set to 1, and the input B is inverted to effectively convert it into its 2's complement, enabling subtraction between A and B. The result of the subtraction is then analyzed to determine the comparison outputs. The output bits of the subtraction are passed through a NOR gate to check if all bits are zero, which indicates that the two numbers are equal. The Less Than (LT) output is determined by comparing the most significant bit (MSB) of the subtraction result with the final carry-out, while the Greater Than (GT) output is obtained by using a NOR gate to combine the Equal and LT outputs, ensuring GT is high only when neither LT nor Equal is true. This method provides an efficient way to compare signed numbers using subtraction and logical gates.

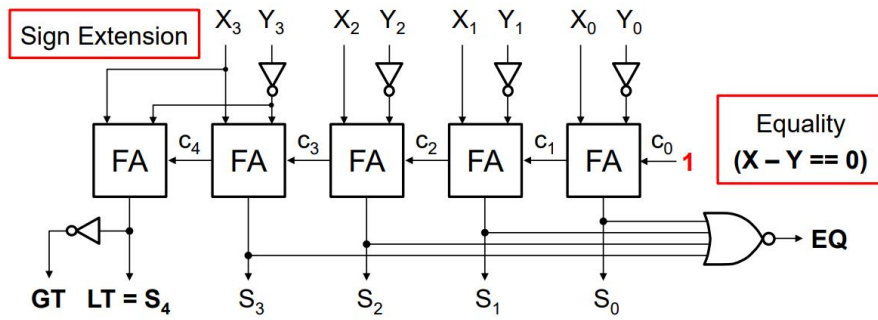


Figure 2: Signed Comparator

The full adder is fully implemented using basic logic gates as follows

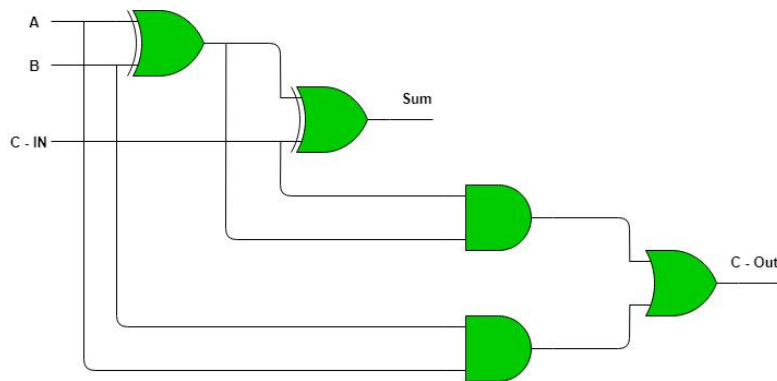


Figure 3: Full adder

Multiplexer

A 2x1 multiplexer (Mux) is used in the design to determine which type of comparator to use: signed or unsigned. A one-bit selection signal is utilized, where 0 selects the unsigned comparator, and 1 selects the signed comparator. The Mux is structurally built using basic gates as follows:

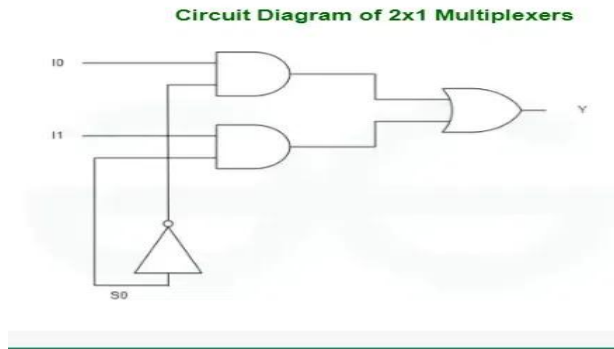


Figure 4:2x1 multiplexer

Where each input is considered an output from the signed and unsigned comparison.

Making the Circuit Synchronous Using Registers

To synchronize the circuit, registers were added to both the inputs and outputs. An additional input, the clock, was introduced to manage the synchronization process. The registers were constructed using D flip-flops that are sensitive to the positive edge of the clock signal.

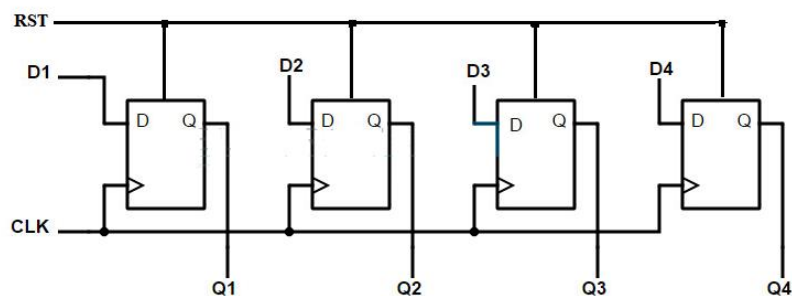


Figure 5: registers

Comparator full design

After combining all parts together using instances of each module and adding registers for the inputs and outputs, the following is the final design block diagram:

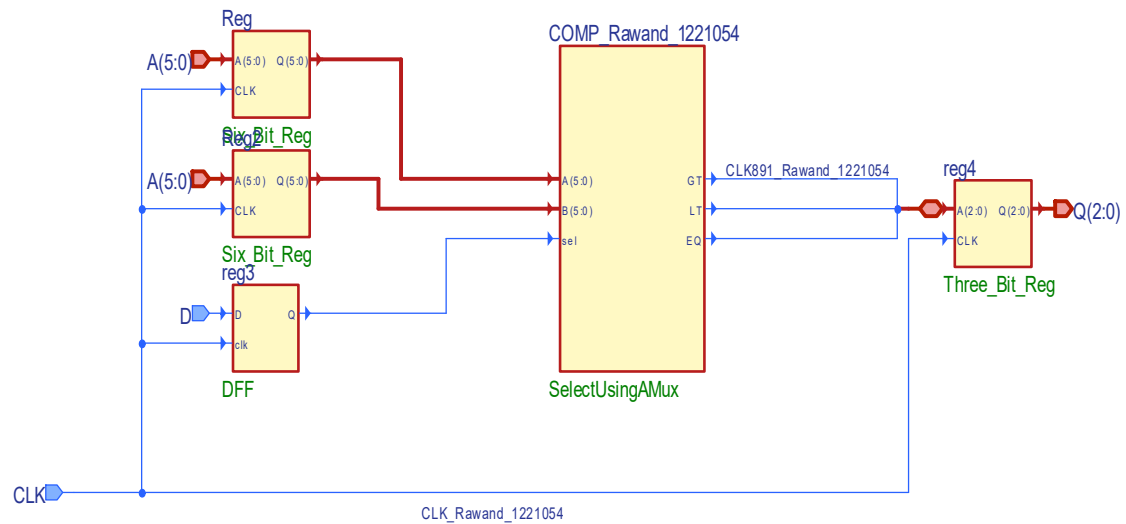


Figure 6: comparator block diagram

In the diagram, the inputs (A, B, and S) are first passed through registers to synchronize them with the clock. The Mux is used to select between the signed or unsigned comparators based on the selection signal S. The output of the selected comparator is then passed through additional registers before the final outputs (Equal, Greater, and Smaller) are generated, ensuring that the design operates in sync with the clock. The delays for each gate are accounted for to maintain proper timing and functionality.

Simulation Results

To test the design's correctness, a testbench was written to test all possible cases for the input numbers. The test process uses nested loops to generate all combinations of the 6-bit input numbers A and B, and 1-bit selection signal S. The correctness of the outputs is verified using conditional if statements, where the expected values for the outputs (Equal, Greater, and Smaller) are compared with the actual values produced by the comparator. If the expected and actual values match,

the test passes; otherwise, an error is flagged. This approach ensures that the comparator behaves correctly for all input combinations and selection states.

An initial 'if' statement was needed for the first inputs of the simulation to avoid the uninitialized input 'X' comparison, since the first input is triggered to the registers at the first rising clock edge, which occurs after half a cycle. Before that, the input is 'X'.

The clock cycle was determined experimentally, starting with a small value where the output was 'x' until it reached a proper value where the output was stable. In this case, the required latency was 110 ns. Since the output needed a latency of 110 ns to appear, the clock cycle time was determined to be equal to this latency. Therefore, the clock frequency can be calculated as:

$$F_{\text{clock}} = \frac{1}{T_{\text{cycle}}}$$

$$F_{\text{clock}} = \frac{1}{110} = 9.09\text{MHz}$$

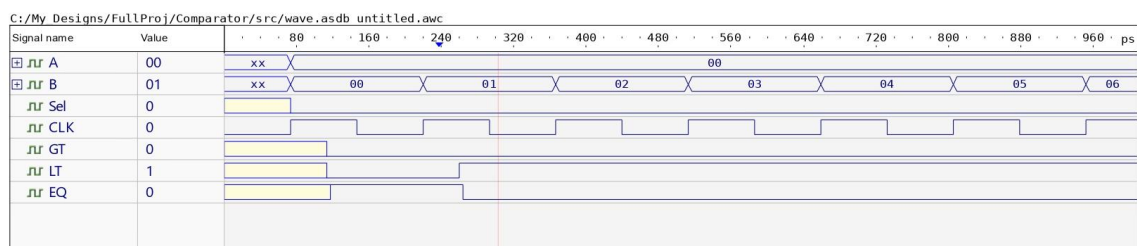


Figure 7:Waveform Result

For further verification of the design, an error was introduced into the design and simulated to check if the test bench would be able to detect it.

An AND gate was replaced with a NAND gate in one element of the design, and the design was simulated. The output was as follows:

```
# KERNEL: ASDB file was created in location C:\My_Designs\FullProj\Comparator\src\wave.asdb
# 2:14 PM, Friday, December 27, 2024
# Simulation has been initialized
# Waveform file 'untitled.awc' connected to 'C:/My_Designs/FullProj/Comparator/src/wave.asdb'.
run
# KERNEL: Test case failed! A = 10000, B = 0, Sel = 0, GT = 0, LT = 0, EQ = 0
# RUNTIME: Info: RUNTIME 0068 Comparator Full Design.v (248): $finish called.
```

Figure 8:Design error dedected

Conclusion and Future works

In conclusion , in this project ,We created a comparator for signed and unsigned six-bit values in this project. "Equal," "Greater," and "Smaller," the comparator's three outputs, matched the comparison outcomes between the two inputs. The comparison was either signed or unsigned based on a selection bit. Simulations were used to validate the design, and the outcomes showed that it worked properly for every possible combination of inputs. We also calculated the highest possible latency and, hence, the highest possible clock frequency.

The project was completed successfully, and the use of test benches for error detection helped guarantee the precision and stability of the design. The use of structural Verilog code allowed for a clear and modular design, which can be easily modified for future improvements.

While the comparator design satisfies the project's requirements, there are a few areas that might need improvement in the future,first,The current design only supports 6-bit inputs. For future work, it would be important to extend the design to support larger bit-widths, such as 8-bit or 16-bit numbers, by generalizing the design using parameters. This would increase the comparator's flexibility and make it suitable for a wider range of applications,An additional improvement that might be useful is the use of advanced gates or more effective techniques to optimize the design, reducing resource costs and improving performance.